- **Prefixes:** and indeed, string prefixes are a key factor for this new data structure, which in turn allows to efficiently perform queries to find strings with a common prefix or, vice versa, given a string finding its longest prefix in the dataset.
- **Radix Trie:** storing tries can be cheaper than holding these values in binary search trees or hash tables, but it can require a lot of memory. Radix tries compress paths, whenever possible, to provide a more compact representation of these tries, without having to compromise in terms of complexity or performance.
- **Applications:** from spell-checkers to bioinformatics, many applications and fields manipulating strings can benefit from using tries. Keep it in mind, when you find yourself writing text-intensive code.

---

[1] Also known as prefix tree.

[2] Also known as radix trie or compact prefix tree.

[3] It has been proven that it's not possible to sort a list of n elements with less than `O(n*log(n))` operations, if the method used is exclusively based on comparisons.

[4] Obviously, it's possible to write spell checkers for any language; choosing English, is just a matter of convenience.

[5] Ring a bell? It should…

[6] With tweets, of course, you can now delete them – if you are fast enough, even before someone notices.

[7] Here the term refers to the abstract data structure called dictionary that, incidentally but not surprisingly, is used to model the digital equivalent of actual dictionaries. See appendix C and chapter 4 for more details.

[8] De La Briandais, Rene. "File searching using variable length keys." Papers presented at the March 3-5, 1959, western joint computer conference. ACM, 1959.

[9] As we'll shortly see, this happens for all characters c for which there is no suffix of current node whose next character is c.